

Packet Capture (libpcap)

Z. Cihan TAYŞI



Outline

- Packet Capturing
- libpcap
- Tools to use

libpcap

- libpcap
 - to find network devices,
 - get information about devices,
 - process packets in real time or offline,
 - send packets,
 - even listen to wireless traffic.
- `sudo apt-get install libpcap-dev`
- `gcc <filename> -lpcap`

First example – List devices

```
#include <stdio.h>
#include <stdlib.h>
#include <pcap.h> /* GIMME a libpcap plz! */
#include <errno.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main(int argc, char **argv)
{
    char *dev; /* name of the device to use */
    char *net; /* dot notation of the network address */
    char *mask; /* dot notation of the network mask */
    int ret; /* return code */
    char errbuf[PCAP_ERRBUF_SIZE];
    bpf_u_int32 netp; /* ip */
    bpf_u_int32 maskp; /* subnet mask */
    struct in_addr addr;

    /* ask pcap to find a valid device for use to sniff on */
    dev = pcap_lookupdev(errbuf);

    /* error checking */
    if(dev == NULL)
    {
        printf("%s\n",errbuf);
        exit(1);
    }

    /* print out device name */
    printf("DEV: %s\n",dev);
```

```
/* ask pcap for the network address and mask of the device */
ret = pcap_lookupnet(dev,&netp,&maskp,errbuf);

if(ret == -1)
{
    printf("%s\n",errbuf);
    exit(1);
}

/* get the network address in a human readable form */
addr.s_addr = netp;
net = inet_ntoa(addr);

if(net == NULL)/* thanks Scott :-P */
{
    perror("inet_ntoa");
    exit(1);
}

printf("NET: %s\n",net);

/* do the same as above for the device's mask */
addr.s_addr = maskp;
mask = inet_ntoa(addr);

if(mask == NULL)
{
    perror("inet_ntoa");
    exit(1);
}

printf("MASK: %s\n",mask);

return 0;
}
```


First example – List devices

- Compilation
 - don't forget to include pcap library!
 - gcc ldev.c -lpcap
- Output
 - differs according to your system
 - Yours might be
 - DEV: eth0
 - NET: 192.168.12.0
 - MASK: 255.255.255.0
 - Mine
 - DEV: en0
 - NET: 192.168.1.0
 - MASK: 255.255.255.0

Capture live traffic

```
#include <stdio.h>
#include <time.h>
#include <pcap.h>
#include <netinet/in.h>
#include <netinet/if_ether.h>

void print_packet_info(const u_char *packet, struct pcap_pkthdr packet_header);

int main(int argc, char *argv[]) {
    char *device;
    char error_buffer[PCAP_ERRBUF_SIZE];
    pcap_t *handle;
    const u_char *packet;
    struct pcap_pkthdr packet_header;
    int packet_count_limit = 1;
    int timeout_limit = 10000; /* In milliseconds */

    device = pcap_lookupdev(error_buffer);
    if (device == NULL) {
        printf("Error finding device: %s\n", error_buffer);
        return 1;
    }

    /* Open device for live capture */
```

```
    /* Open device for live capture */
    handle = pcap_open_live(
        device,
        BUFSIZ,
        packet_count_limit,
        timeout_limit,
        error_buffer
    );

    /* Attempt to capture one packet. If there is no network traffic
    and the timeout is reached, it will return NULL */
    packet = pcap_next(handle, &packet_header);
    if (packet == NULL) {
        printf("No packet found.\n");
        return 2;
    }

    /* Our function to output some info */
    print_packet_info(packet, packet_header);

    return 0;
}
```


Capture live traffic

```
void print_packet_info(const u_char *packet, struct pcap_pkthdr packet_header) {  
    printf("Packet capture length: %d\n", packet_header.caplen);  
    printf("Packet total length %d\n", packet_header.len);  
}
```

We can include more info !
TIME

```
struct pcap_pkthdr {  
    struct timeval ts;           /* time stamp */  
    bpf_u_int32 caplen;          /* length of portion present */  
    bpf_u_int32 len;             /* length this packet (off wire) */  
};
```

example in live.c

use

- (1) localtime() to convert from timeval to struct tm
- &
- (2) strftime() to print time in human readable form

Capturing more packets

```
#include <pcap.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netinet/if_ether.h>

/* callback function that is passed to pcap_loop(..) and called each time
 * a packet is recieved */
void my_callback(u_char *useless, const struct pcap_pkthdr* pkthdr, const u_char* packet)
{
    static int count = 1;
    fprintf(stdout, "%d, ", count);
    if(count == 4)
        fprintf(stdout, "Come on baby sayyy you love me!!! ");
    if(count == 7)
        fprintf(stdout, "Tiiimmmmeesss!! ");
    fflush(stdout);
    count++;
}
```

```
int pcap_loop(pcap_t *p, int cnt, pcap_handler callback, u_char *user)
```

This is how our callback function must look like

```
/* The contract we have to satisfy with our callback function */
typedef void (*pcap_handler)(u_char *, const struct pcap_pkthdr *,
                             const u_char *);
```


Capturing more packets

- **pcap_t** is the device handle from where we want to capture.
- The second argument is an int which is the number of packets you want to capture.
 - Pass **0** for unlimited packets.
- The **pcap_handler** accepts a function name that is the callback to be run on every packet captured.
- The last argument to **pcap_loop** is arguments to the callback function.

```
int main(int argc, char **argv)
{
    int i;
    char *dev;
    char errbuf[PCAP_ERRBUF_SIZE];
    pcap_t* descr;
    const u_char *packet;
    struct pcap_pkthdr hdr;    /* pcap.h */
    struct ether_header *eptr; /* net/ethernet.h */

    if(argc != 2){ fprintf(stdout, "Usage: %s numpackets\n", argv[0]); return 0; }

    /* grab a device to peak into... */
    dev = pcap_lookupdev(errbuf);
    if(dev == NULL)
    { printf("%s\n", errbuf); exit(1); }
    /* open device for reading */
    descr = pcap_open_live(dev, BUFSIZ, 0, -1, errbuf);
    if(descr == NULL)
    { printf("pcap_open_live(): %s\n", errbuf); exit(1); }

    /* allright here we call pcap_loop(..) and pass in our callback function */
    /* int pcap_loop(pcap_t *p, int cnt, pcap_handler callback, u_char *user) */
    /* If you are wondering what the user argument is all about, so am I!! */
    pcap_loop(descr, atoi(argv[1]), my_callback, NULL);

    fprintf(stdout, "\nDone processing packets... wheew!\n");
    return 0;
}
```

loop_ex1.c

Determining Packet Type

```
#include <stdio.h>
#include <pcap.h>
#include <netinet/in.h>
#include <net/ethernet.h>

/* This function can be used as a callback for pcap_loop() */
void my_packet_handler(
    u_char *args,
    const struct pcap_pkthdr* header,
    const u_char* packet
) {
    struct ether_header *eth_header;
    /* The packet is larger than the ether_header struct,
       but we just want to look at the first part of the packet
       that contains the header. We force the compiler
       to treat the pointer to the packet as just a pointer
       to the ether_header. The data payload of the packet comes
       after the headers. Different packet types have different header
       lengths though, but the ethernet header is always the same (14 bytes) */
    eth_header = (struct ether_header *) packet;

    if (ntohs(eth_header->ether_type) == ETHERTYPE_IP) {
        printf("IP\n");
    } else if (ntohs(eth_header->ether_type) == ETHERTYPE_ARP) {
        printf("ARP\n");
    } else if (ntohs(eth_header->ether_type) == ETHERTYPE_REVARP) {
        printf("Reverse ARP\n");
    }
}
```

- compile and run the example
 - you will see IP on each new line
- open a new terminal
 - ping some address
- you should start seeing
 - ARP !

loop_ex2.c

Loading a Pcap file

```
pcap_t *pcap_open_offline(const char *fname, char *errbuf);
```

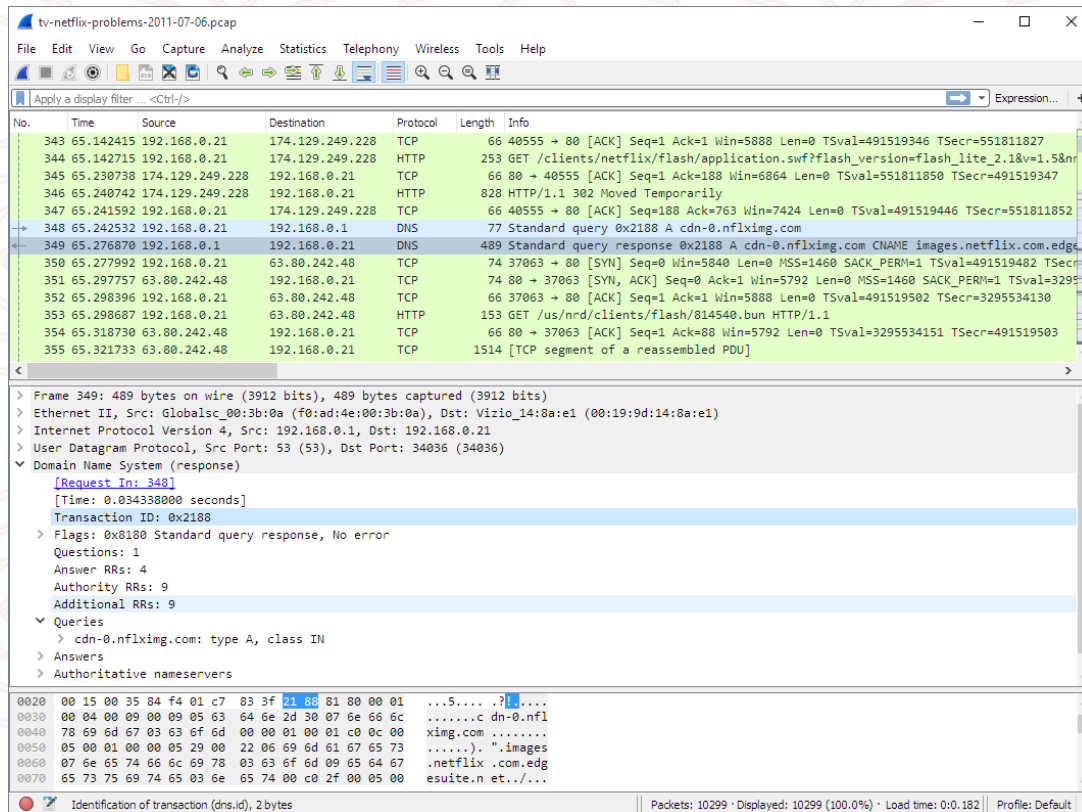
```
char error_buffer[PCAP_ERRBUF_SIZE];  
pcap_t *handle = pcap_open_offline("capture_file.pcap", error_buffer);
```

- Loading a pcap file is just like opening a device. Opening a file returns a **pcap_t** just like opening a network device.

Wireshark

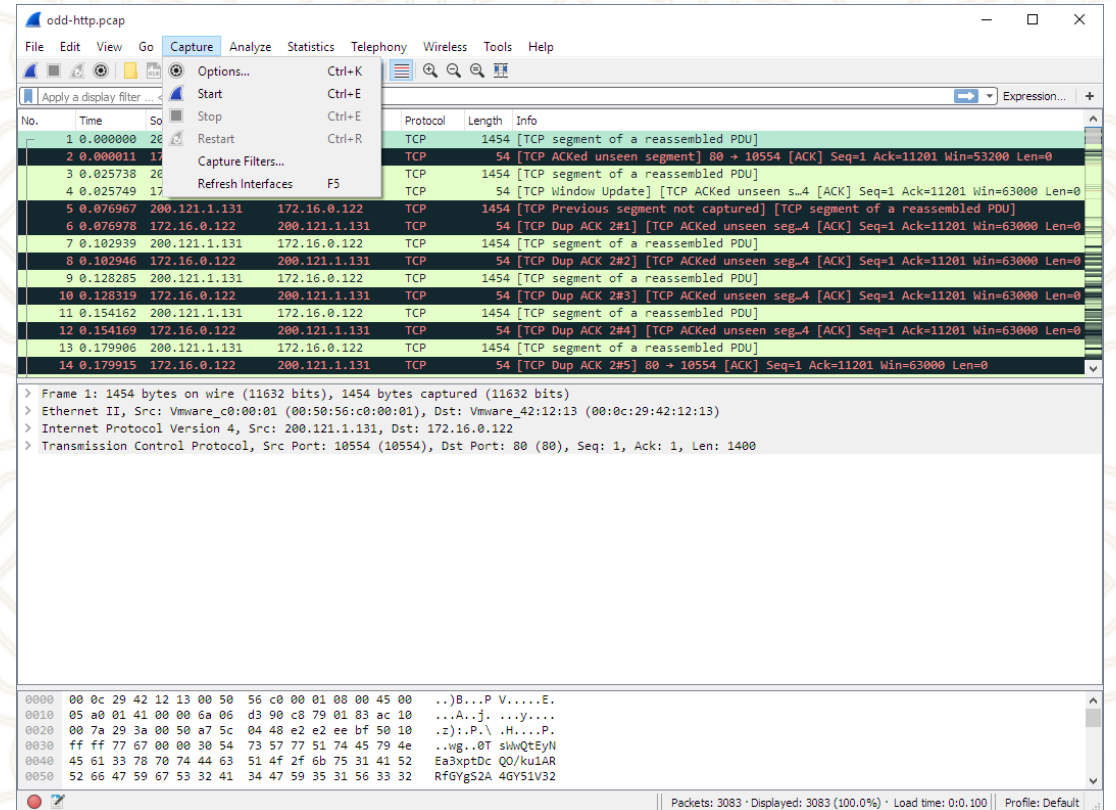
- Wireshark is a network packet analyzer. A network packet analyzer presents captured packet data in as much detail as possible.
 - live capture
 - import files
- Where to get
 - <https://www.wireshark.org/download.html>

Wireshark – Main Menu

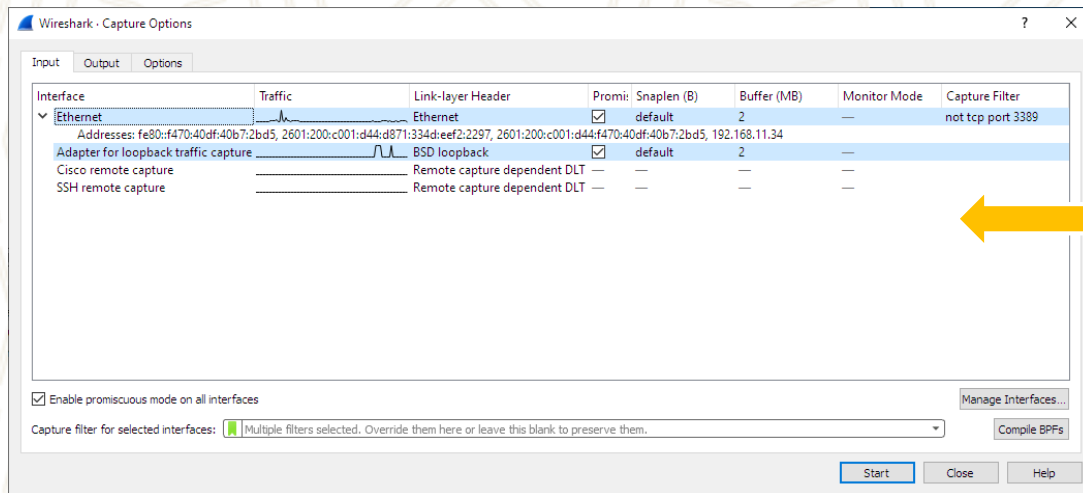


1. The **menu** is used to start actions.
2. The **main toolbar** provides quick access to frequently used items from the menu.
3. The **filter toolbar** allows users to set *display filters* to filter which packets are displayed
4. The **packet list pane** displays a summary of each packet captured. By clicking on packets in this pane you control what is displayed in the other two panes.
5. The **packet details pane** displays the packet selected in the packet list pane in more detail.
6. The **packet bytes pane** displays the data from the packet selected in the packet list pane, and highlights the field selected in the packet details pane.
7. The **statusbar** shows some detailed information about the current program state and the captured data.

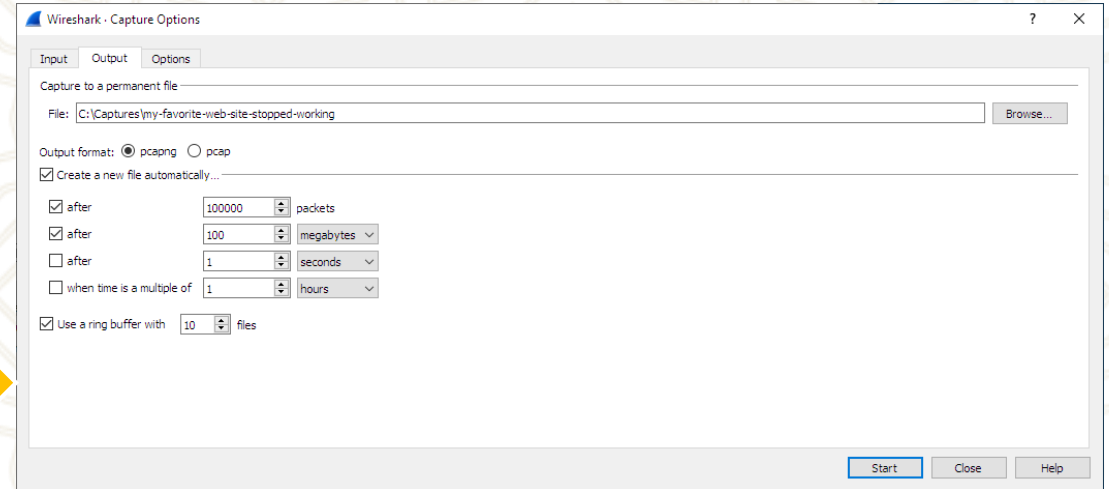
Wireshark – Capture Menu



Wireshark – Capture Menu



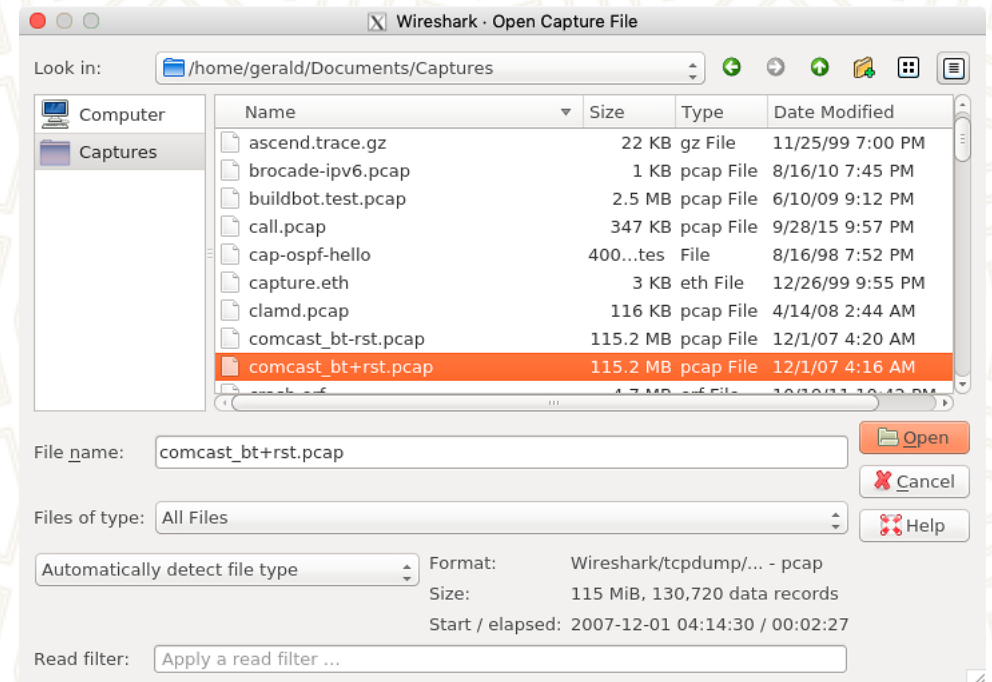
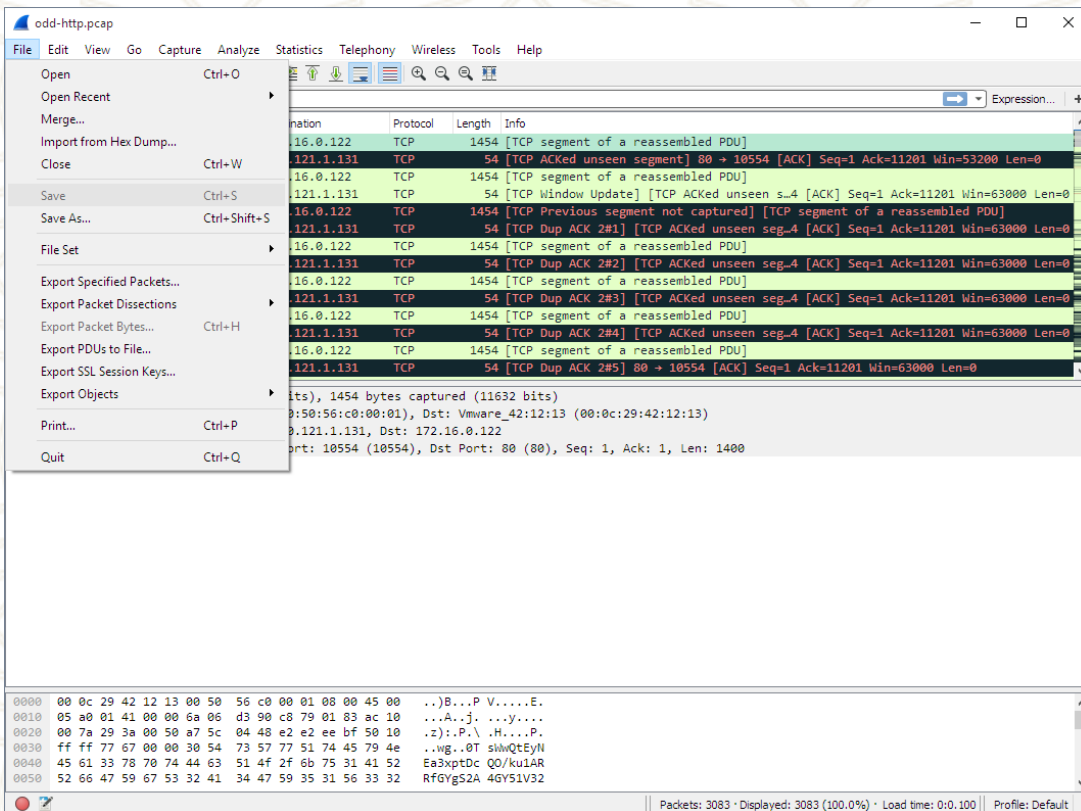
If you are unsure which options to choose in this dialog box, leaving the defaults settings as they are should work well in many cases.



By default Wireshark saves packets to a temporary file. You can also tell Wireshark to save to a specific (“permanent”) file and switch to a different file after a given time has elapsed or a given number of packets have been captured

Wireshark – File Menu

The “Open Capture File” dialog box allows you to search for a capture file containing previously captured packets for display in Wireshark.

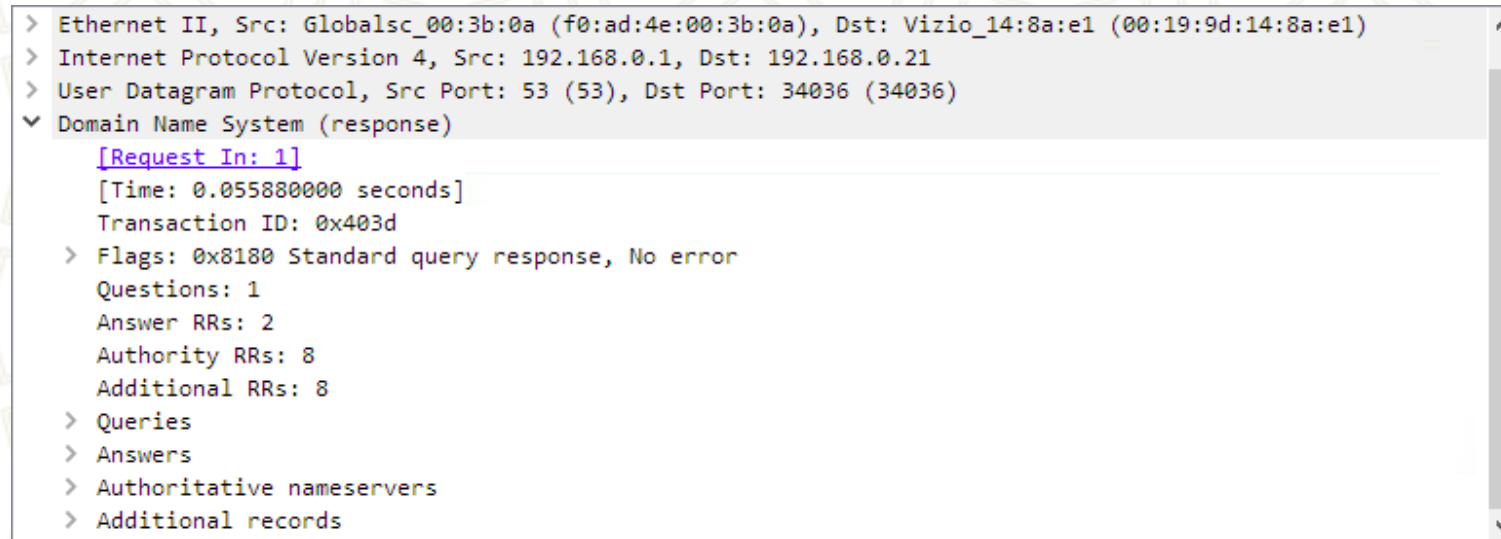


Wireshark – The “Packet List” Pane

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.0.21	192.168.0.1	DNS	84	Standard query 0x403d A moviecontrol.netflix.com
2	0.055880	192.168.0.1	192.168.0.21	DNS	479	Standard query response 0x403d A moviecontrol.netflix.com CNAME nccp-moviecontrol-fron
3	0.057690	192.168.0.21	50.17.249.22	TCP	74	37314→443 [SYN] Seq=0 Win=5840 Len=0 MSS=1460 SACK_PERM=1 TSval=491454310 TSecr=0 WS=6
4	0.154716	50.17.249.22	192.168.0.21	TCP	74	443→37314 [SYN, ACK] Seq=0 Ack=1 Win=5792 Len=0 MSS=1460 SACK_PERM=1 TSval=2102931926
5	0.155962	192.168.0.21	50.17.249.22	TCP	66	37314→443 [ACK] Seq=1 Ack=1 Win=5888 Len=0 TSval=491454408 TSecr=2102931926
6	0.163169	192.168.0.21	50.17.249.22	TLSv1	187	Client Hello
7	0.250734	50.17.249.22	192.168.0.21	TCP	66	443→37314 [ACK] Seq=1 Ack=122 Win=5792 Len=0 TSval=2102931950 TSecr=491454416
8	0.252716	50.17.249.22	192.168.0.21	TLSv1	1514	Server Hello
9	0.253826	192.168.0.21	50.17.249.22	TCP	66	37314→443 [ACK] Seq=122 Ack=1449 Win=8768 Len=0 TSval=491454507 TSecr=2102931950
10	0.254730	50.17.249.22	192.168.0.21	TCP	1514	[TCP segment of a reassembled PDU]
11	0.254778	50.17.249.22	192.168.0.21	TLSv1	349	Certificate
12	0.255853	192.168.0.21	50.17.249.22	TCP	66	37314→443 [ACK] Seq=122 Ack=2897 Win=11648 Len=0 TSval=491454509 TSecr=2102931950
13	0.256102	192.168.0.21	50.17.249.22	TCP	66	37314→443 [ACK] Seq=122 Ack=3180 Win=14528 Len=0 TSval=491454509 TSecr=2102931950
14	0.319870	192.168.0.21	50.17.249.22	TLSv1	264	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
15	0.411795	50.17.249.22	192.168.0.21	TLSv1	125	Change Cipher Spec, Encrypted Handshake Message

- Each line in the packet list corresponds to one packet in the capture file.
- If you select a line in this pane, more details will be displayed in the “Packet Details” and “Packet Bytes” panes.

Wireshark – The “Packet Details” Pane



- This pane shows the protocols and protocol fields of the packet selected in the “Packet List” pane.
- The protocols and fields of the packet shown in a tree which can be expanded and collapsed.

Wireshark – The “Packet Bytes” Pane

0000	00 19 9d 14 8a e1 f0 ad 4e 00 3b 0a 08 00 45 00	 N.;...E.
0010	01 d1 00 00 40 00 40 11 b7 b5 c0 a8 00 01 c0 a8	@.@.
0020	00 15 00 35 84 f4 01 bd 83 35 40 3d 81 80 00 01	...5.... .5@=....
0030	00 02 00 08 00 08 0c 6d 6f 76 69 65 63 6f 6e 74	m oviecont
0040	72 6f 6c 07 6e 65 74 66 6c 69 78 03 63 6f 6d 00	rol.netf lix.com.
0050	00 01 00 01 c0 0c 00 05 00 01 00 00 00 2d 00 40	-.@
0060	25 6e 63 63 70 2d 6d 6f 76 69 65 63 6f 6e 74 72	%nccp-mo viecontr
0070	6f 6c 2d 66 72 6f 6e 74 65 6e 64 2d 31 37 31 32	ol-front end-1712
0080	31 38 38 39 32 31 09 75 73 2d 65 61 73 74 2d 31	188921.u s-east-1
0090	03 65 6c 62 09 61 6d 61 7a 6f 6e 61 77 73 c0 21	.elb.ama zonaws.!

- The packet bytes pane shows the data of the current packet (selected in the “Packet List” pane) in a hexdump style.
- Each line contains the data offset, sixteen hexadecimal bytes, and sixteen ASCII bytes. Non-printable bytes are replaced with a period (“.”)

Assignment

- Write a program to find and list all ICMP packets
 - Your program should capture live traffic
 - You can **not** use filter mechanism!
 - Your program will analyse and find ICMP packets
 - Your program should list
 - date
 - source IP
 - destination IP
 - ICMP packet type

Hints

- tcpdump is there, just grab the source and take a look at the following methods !
 - ether_if_print (print-ether.c)
 - ip_print (print-ip.c)
 - tcp_print (print-tcp.c)
 - udp_print (print-udp.c)
- Wireless,
 - Promiscuous, and Monitor Mode
- use of filters

References & More to read

- <http://yuba.stanford.edu/~casado/pcap/section1.html>
- https://www.wireshark.org/docs/wsug_html_chunked/ChapterIntroduction.html#ChIntroWhatIs
- <https://www.devdungeon.com/content/using-libpcap-c#packet-info>